DATA ENGINEERING • MODULE 05

Kafka & Streaming

Real-Time Data Pipelines at Scale

■ STREAMING

■ LEARNING OUTCOME

By the end of this module you will understand Kafka's architecture, produce and consume events in Python, build a streaming pipeline with Spark Structured Streaming, and apply windowing and watermarking for time-based aggregations.

■ Skills You Will Gain

- Explain Kafka architecture: brokers, topics, partitions, consumer groups
- Produce and consume messages with kafka-python
- Build a streaming pipeline with Spark Structured Streaming
- Apply tumbling and sliding windows with watermarking for late data
- Understand exactly-once, at-least-once, and at-most-once delivery
- Monitor Kafka with consumer lag and choose streaming vs batch

■ **Real-time streaming is the fastest growing DE skill. Companies pay 30% more for engineers who build streaming pipelines. Kafka powers LinkedIn, Netflix, Uber, and every major tech company.**

SECTION 1

Kafka Architecture

Concept	Description
Topic	Named channel for messages — like a database table, but append-only and ordered
Partition	Ordered, immutable log of messages. More partitions = more parallelism. Messages within a
Broker	A Kafka server node that stores partitions and handles reads/writes from producers and con
Producer	Client that publishes messages to topics. Can choose partition by key hash for ordering gua
Consumer	Client that subscribes to topics and reads messages sequentially, tracking offset position
Consumer Group	Multiple consumers sharing partitions — each partition assigned to exactly ONE consumer i
Offset	Sequential integer position of a message in a partition. Consumers commit offsets to track p
Retention	How long Kafka stores messages — default 7 days. Can replay from any offset within retent

Delivery Guarantees — Choose the Right Trade-off

At-most-once: Messages may be lost but never duplicated. Simple, fast, use for metrics/logs. At-least-once: Messages may be duplicated but never lost. Requires idempotent consumers. Most common. Exactly-once: Each message processed exactly once. Requires Kafka transactions + idempotent producers. Complex. Producer config for at-least-once: acks='all', retries=3, enable.idempotence=True

SECTION 2

Produce and Consume with Python

```
from kafka import KafkaProducer, KafkaConsumer import json # Producer — send events to Kafka topic producer = KafkaProducer( bootstrap_servers=['localhost:9092'], value_serializer=lambda v: json.dumps(v).encode('utf-8'), key_serializer=lambda k: k.encode('utf-8'), acks='all', # wait for all replicas to confirm retries=3, enable_idempotence=True # exactly-once semantics at producer ) # Send an event order_event = { 'order_id': 1001, 'user_id': 42, 'amount': 199.99, 'status': 'placed', 'timestamp': '2024-01-15T10:30:00Z' } producer.send( topic='orders', key=f'user_{order_event["user_id"]}', # same user -> same partition -> ordered value=order_event ) producer.flush() # ensure delivery before exit # Consumer — read events from Kafka topic consumer = KafkaConsumer( 'orders', bootstrap_servers=['localhost:9092'], group_id='order-processor-v1', auto_offset_reset='earliest', # start from beginning if no committed offset value_deserializer=lambda m: json.loads(m.decode('utf-8')), enable_auto_commit=True ) for message in consumer: event = message.value partition, offset = message.partition, message.offset print(f'[P{partition}:{offset}] Order {event["order_id"]}: ${event["amount"]}')
```

SECTION 3

Spark Structured Streaming

```
from pyspark.sql import SparkSession from pyspark.sql import functions as F from pyspark.sql.types import StructType, StructField, IntegerType, DoubleType, TimestampType spark = SparkSession.builder.appName('OrderStreaming').getOrCreate() # Define schema for JSON messages order_schema = StructType([ StructField('order_id', IntegerType()), StructField('user_id', IntegerType()), StructField('amount', DoubleType()), StructField('timestamp', TimestampType()), ])
```

```
# Read from Kafka as a streaming DataFrame stream_df = ( spark.readStream .format('kafka')
.option('kafka.bootstrap.servers', 'localhost:9092') .option('subscribe', 'orders')
.option('startingOffsets', 'latest') .load() .select(F.from_json(F.col('value').cast('string'),
order_schema).alias('data')) .select('data.*') ) # Windowed aggregation with watermark for late
data windowed = ( stream_df .withWatermark('timestamp', '10 minutes') # allow 10min late arrival
.groupBy(F.window('timestamp', '1 hour', '10 minutes')) # 1h window, 10m slide .agg(
F.sum('amount').alias('revenue'), F.count('*').alias('order_count') ) ) # Write to console (use
parquet/kafka sink in production) query = ( windowed.writeStream .outputMode('update') # emit only
updated windows .format('console') .option('truncate', False) .trigger(processingTime='30 seconds')
# micro-batch every 30s .start() ) query.awaitTermination()
```

SECTION 4

Streaming Concepts Reference

Concept	Explanation
Tumbling Window	Fixed-size, non-overlapping time windows. window('ts','1 hour') — 00:00-01:00, 01:00-02:00
Sliding Window	Overlapping windows. window('ts','1 hour','15 minutes') — windows slide every 15 min
Session Window	Gaps of inactivity close the window. Groups user activity bursts
Watermark	Maximum allowed lateness for data. Older data is dropped. Controls state size and output c
Consumer Lag	Latest_offset - consumer_current_offset. High lag = consumer can't keep up with production
Checkpointing	Periodically save streaming query state to fault-tolerant storage. Enables recovery after failu
Exactly-once sink	Use idempotent writes (upsert) so replayed events don't create duplicates

■ PRACTICE Q & A

Q1. What is a Kafka consumer group and why does it enable horizontal scaling?

A: *A set of consumers sharing consumption of a topic. Each partition is assigned to exactly ONE consumer in the group. Add more consumers to read more partitions in parallel — linear horizontal scaling.*

Q2. What is the difference between at-least-once and exactly-once delivery?

A: *At-least-once: messages guaranteed delivered but may be duplicated on retry — requires idempotent consumers. Exactly-once: each message processed exactly once — requires Kafka transactions (acks='all', enable.idempotence=True).*

Q3. What is watermarking in Spark Structured Streaming?

A: *Defines how late data can arrive. Data older than watermark is dropped. Allows Spark to finalise and clean up windowed state, preventing unbounded memory growth from late-arriving events.*

Q4. When would you choose Kafka over a database message queue (like RabbitMQ)?

A: *Kafka for: high throughput (millions/sec), long retention and replay, multiple independent consumers, stream processing. RabbitMQ for: simpler task queuing, lower volume, routing logic, when you need message acknowledgement patterns.*

Q5. What is consumer lag and why does it matter?

A: *Difference between latest message offset and consumer's current committed offset. High lag means the consumer is falling behind production rate. Monitor with Kafka consumer lag metric — alert if lag grows continuously.*

■ Kafka Cheat Sheet	
`KafkaProducer(bootstrap_servers)`	Create producer
`producer.send(topic, key, value)`	Send message to topic
`producer.flush()`	Ensure all sent before exit
`KafkaConsumer(topic, group_id)`	Create consumer

`auto_offset_reset='earliest'`	Read from beginning
`acks='all'`	Strongest delivery guarantee
`enable_idempotence=True`	No duplicate messages
`readStream.format('kafka')`	Spark streaming source
`withWatermark('ts', '10 minutes')`	Allow 10min late data
`window('ts','1 hour')`	1-hour tumbling window
`outputMode('update')`	Emit only changed results
`query.awaitTermination()`	Keep streaming query running